College Placement Portal Using AWS

Prepared in the partial fulfillment of the Summer Internship Program on AWS

AT

Under the guidance of

Mrs. Bethala Sumana, APSSDC

Mr. Juttuka Lovababu, APSSDC

Submitted by

PARIMI LAKSHMI VAMSI, 23BQ1A05H0, VVIT Guntur, (Batch -1)

PUTLA DANY ARSHITH, 23BQ1A05I9, VVIT Guntur, (Batch -1)

MYLA SAI HARSHITH, 23BQ1A05F0, VVIT Guntur, (Batch -1)

REDDY NAGA VIGNESH, 23BQ1A05J5, VVIT Guntur, (Batch -2)

MIRIYALA SRI RAJA YASWANTH, 23BQ1A05E0, VVIT Guntur, (Batch -2)

ACKNOWLEDGEMENT

We would like to express our heartfelt gratitude to all those who have contributed to the successful completion of our summer internship project at Andhra Pradesh Skill Development Corporation (APSSDC). This opportunity has been an enriching and transformative experience for us, and we are truly thankful for the support, guidance, and encouragement we have received along the way.

We extend our sincere regards to Mrs. Bethala Sumana and Mr. Juttuka Lovababu, our mentors, for providing us with valuable insights, constant guidance, and unwavering support throughout the duration of the internship. Their expertise and encouragement have been instrumental in shaping the direction of this project.

We would also like to thank the entire team at Andhra Pradesh Skill Development Corporation (APSSDC) for fostering a collaborative and innovative environment. The camaraderie, knowledge sharing, and feedback we received from our peers significantly contributed to the development and success of this project.

In conclusion, we are honoured to have been a part of this internship program, and we look forward to leveraging the skills and knowledge gained to contribute positively to future endeavours.

Thank you.

Sincerely,

PARIMI LAKSHMI VAMSI
PUTLA DANY ARSHITH
MYLA SAI HARSHITH
REDDY NAGA VIGNESH
MIRIYALA SRI RAJA YASWANTH

ABSTRACT

This report presents the design, development, and cloud deployment of the College Placement Portal, a full-stack web application built to streamline campus recruitment for three types of users: Students, Alumni, and Placement Administrators. The platform enables students to register, build their profile, browse job openings posted by recruiters, upload resumes, and track the real-time status of their applications from "Applied" through "Shortlisted" to "Accepted". Placement officers can add companies, publish job postings, review applications, and manage candidate pipelines from a dedicated admin dashboard.

The application follows a modern three-tier architecture. The frontend is a single-page application built with HTML, CSS, and JavaScript, hosted as a static website on Amazon S3. The backend is a Node.js and Express REST API deployed on an Amazon EC2 instance, secured with JSON Web Token (JWT) based authentication using HS512-signed tokens. Application data — student profiles, company records, job listings, and applications — is stored in MongoDB Atlas, while uploaded files such as resumes and profile photos are stored in a dedicated Amazon S3 bucket. AWS Identity and Access Management (IAM) governs least-privilege access between services.

Development involved resolving several real-world engineering challenges, including reconciling Lombok-generated and manually written constructors, correcting mismatches between frontend API calls and backend route definitions, fixing JWT secret key sizing to satisfy HS512 requirements, and restructuring the authentication service to prevent partial database commits when key errors occurred. The result is a stable, cloud-hosted, multi-role placement management system that demonstrates the practical application of full-stack development and AWS cloud deployment to a real institutional problem.

TABLE OF CONTENTS

1. INTRODUCTION

Campus placement drives at most colleges are still coordinated through a patchwork of spreadsheets, email threads, and notice boards. This makes it difficult for students to discover relevant openings quickly, for placement officers to track application pipelines across many companies, and for anyone to get a clear, real-time picture of where each candidate stands in the process. The College Placement Portal was built to replace this fragmented workflow with a single, cloud-hosted web application that gives every stakeholder — student, alumnus, and placement administrator — a live, role-appropriate view of the recruitment process.

The system is designed around three user roles. Students register using their college email address, complete their profile, browse job listings, apply with a resume upload, and track their application status. Alumni can access a lighter-weight version of the same portal to stay connected with placement activity and mentor current students. Placement Administrators manage the master data of the system: they onboard recruiting companies, publish job postings with eligibility criteria and deadlines, and move applications through the pipeline as interviews and offers progress.

Architecturally, the project demonstrates how a full-stack setup using MongoDB, Express, Node.js, and an HTML, CSS, and JavaScript frontend can be deployed on AWS infrastructure rather than a generic hosting provider, gaining the benefits of scalable file storage (Amazon S3), elastic compute (Amazon EC2), fine-grained access control (AWS IAM), and an isolated network environment (Amazon VPC). This report documents the methodology, technology stack, system architecture, implementation details, and results of building and deploying the College Placement Portal.

2. METHODOLOGY

The development of the College Placement Portal followed an iterative, feature-driven methodology spanning requirements gathering, system design, implementation, testing, deployment, and refinement. Each phase is summarized below.

2.1 Requirements Gathering

The core requirement was a multi-role placement platform supporting Students, Alumni, and Admins, with secure registration, job discovery, resume-based applications, and status tracking. Security requirements included secure account activation and token-based session management using JWT.

2.2 Design

A three-tier, cloud-native architecture was designed: a frontend built with HTML, CSS, and JavaScript served as a static website from Amazon S3, a Node.js/Express REST API running on an Amazon EC2 instance within an Amazon VPC, and MongoDB Atlas as the primary data store. Amazon S3 was additionally used as a dedicated file store for resumes, profile photos, and company logos, decoupling file storage from the application database.

2.3 Implementation

Backend implementation centered on Spring Boot-style layered services adapted to a Node.js/Express structure: controllers for each role (student, alumni, admin), a UserService and AuthService for registration and login, a JwtUtil for token issuance and validation, and a JwtAuthenticationFilter equivalent middleware to protect authenticated routes. On the frontend, JavaScript modules such as registration.js and login.js handled the registration and authentication flows, while dedicated dashboard and job-listing scripts rendered role-specific views.

2.4 Database Management

MongoDB Atlas was chosen for its managed, horizontally scalable NoSQL model, well suited to the loosely structured, evolving schema of student profiles, job postings, and applications. Collections were created for students, companies, jobs, applications, and admins, with each application document referencing the related student and job records.

2.5 Testing

Testing covered both backend and frontend layers. On the backend, endpoint testing via Postman validated registration, login, company creation, job creation, job application, and resume upload flows. On the frontend, each role's dashboard and workflow was manually verified against the corresponding backend routes to catch and correct endpoint mismatches between the two layers.

2.6 Deployment

The frontend build was uploaded to an Amazon S3 bucket configured for static website hosting, while the backend was deployed to an Amazon EC2 instance running Node.js under PM2 for process management and Nginx as a reverse proxy. IAM roles were attached to the EC2 instance to grant least-privilege access to S3 and other AWS resources, and the EC2 instance was placed inside a dedicated Amazon VPC with a security group restricting inbound traffic to SSH and web ports.

2.7 Iterative Refinement

Several rounds of debugging refined the system: reconciling Lombok-generated constructors with manually written ones, correcting route mismatches between frontend API calls and backend endpoints, resizing the JWT signing secret to meet HS512 requirements, and restructuring AuthService so that a bad key error could no longer leave a partial user record committed to the database.

3. TECHNOLOGY STACK

The College Placement Portal was built using the following technologies and AWS services:

3.1 Frontend

- HTML, CSS, and JavaScript — core web technologies used to build the single-page application
- Axios / Fetch API — communication with the backend REST API
- Amazon S3 (Static Website Hosting) — hosts the built HTML, CSS, and JavaScript application

3.2 Backend

- Node.js with Express — REST API framework
- JSON Web Tokens (JWT, HS512) — stateless authentication
- PM2 — process manager keeping the Node.js API alive on EC2
- Nginx — reverse proxy in front of the Node.js application

3.3 Database & Storage

- MongoDB Atlas — managed NoSQL database for students, companies, jobs, applications, and admins
- Amazon S3 — object storage for resumes, profile photos, and company logos

3.4 AWS Cloud Services

- Amazon EC2 — hosts the Node.js/Express backend API
- Amazon S3 — static frontend hosting and file storage
- AWS IAM — least-privilege roles and permissions for EC2 and connected services
- Amazon VPC — isolated network environment with security groups controlling inbound traffic to the EC2 instance

3.5 Tooling

- Postman — API endpoint testing
- Git — version control

4. SYSTEM DESIGN / ARCHITECTURE

The College Placement Portal follows a three-tier, cloud-hosted architecture that separates presentation, application logic, and data persistence. Amazon S3 serves the static HTML, CSS, and JavaScript frontend and stores uploaded files; Amazon EC2, running inside an Amazon VPC, hosts the Node.js/Express REST API; MongoDB Atlas persists structured application data; and AWS IAM together with the Amazon VPC network boundary provide security across the stack.

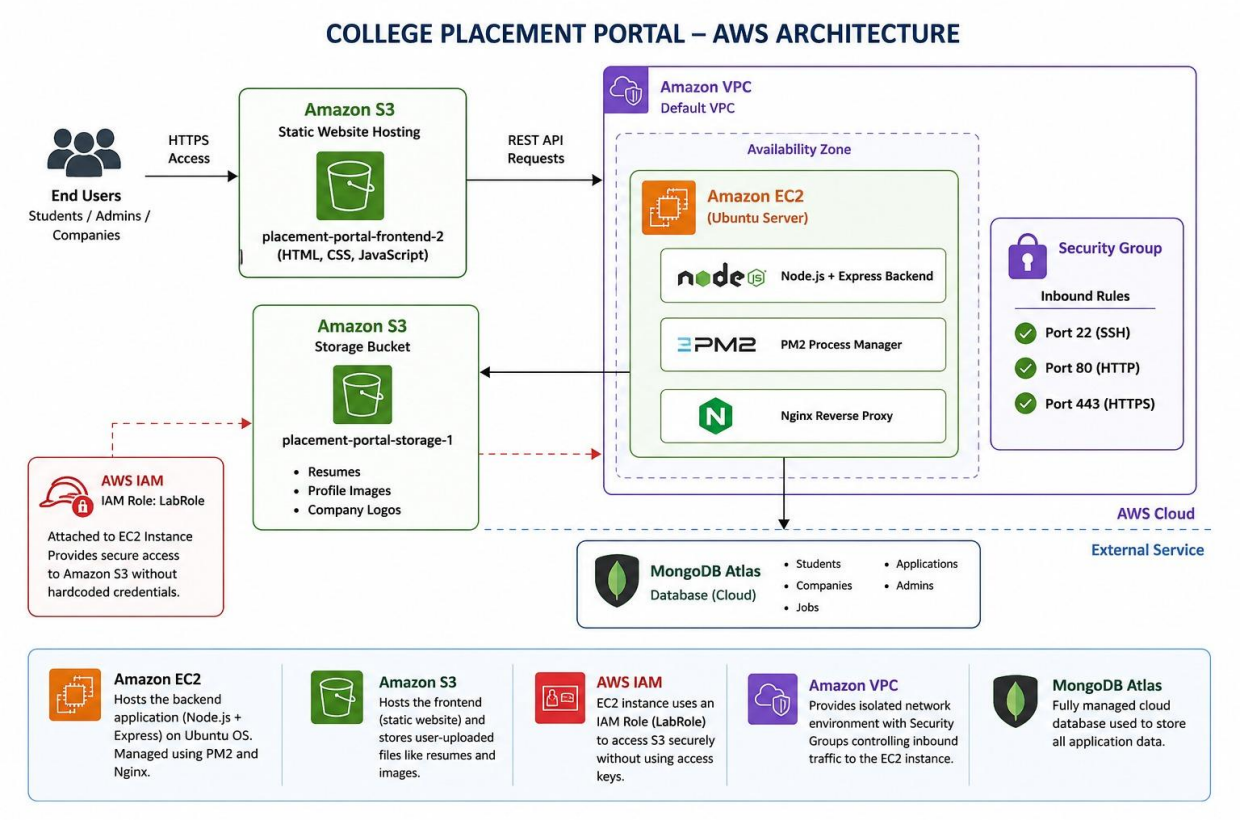

Figure 1. College Placement Portal — AWS Deployment Architecture

4.1 Presentation Layer — Amazon S3 (HTML, CSS, and JavaScript Frontend)

The HTML, CSS, and JavaScript application is built into static assets and uploaded to an Amazon S3 bucket configured for static website hosting. This gives students, alumni, and admins a fast, globally accessible landing page from which every role-specific workflow begins.

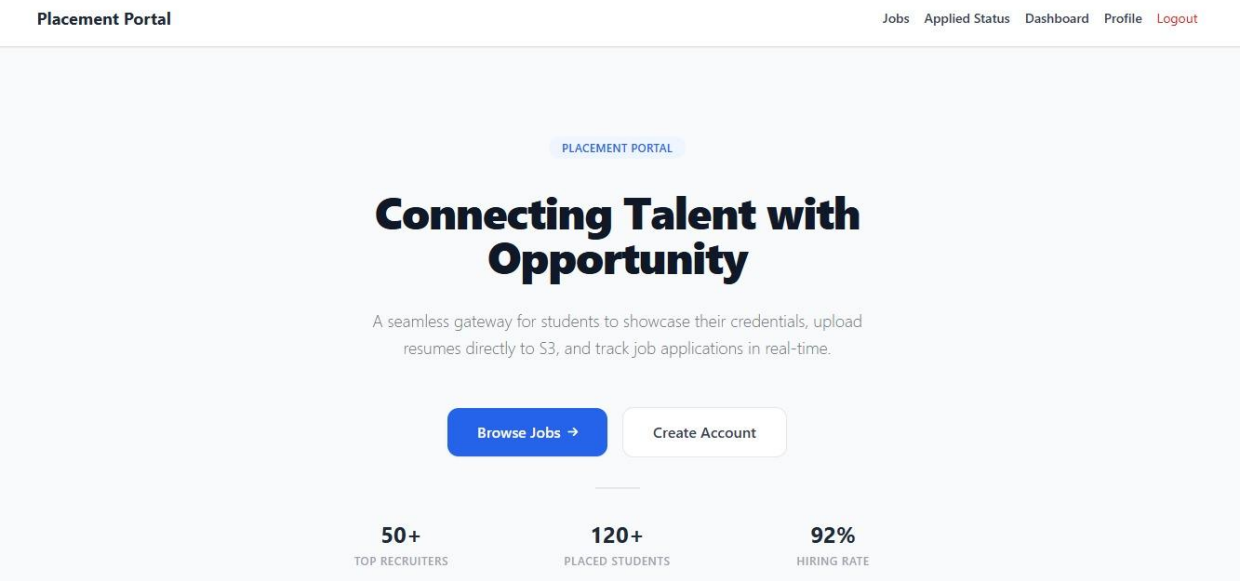

Figure 2. Placement Portal Homepage Hosted on Amazon S3

New students create an account through a registration form that collects name, email, and password.

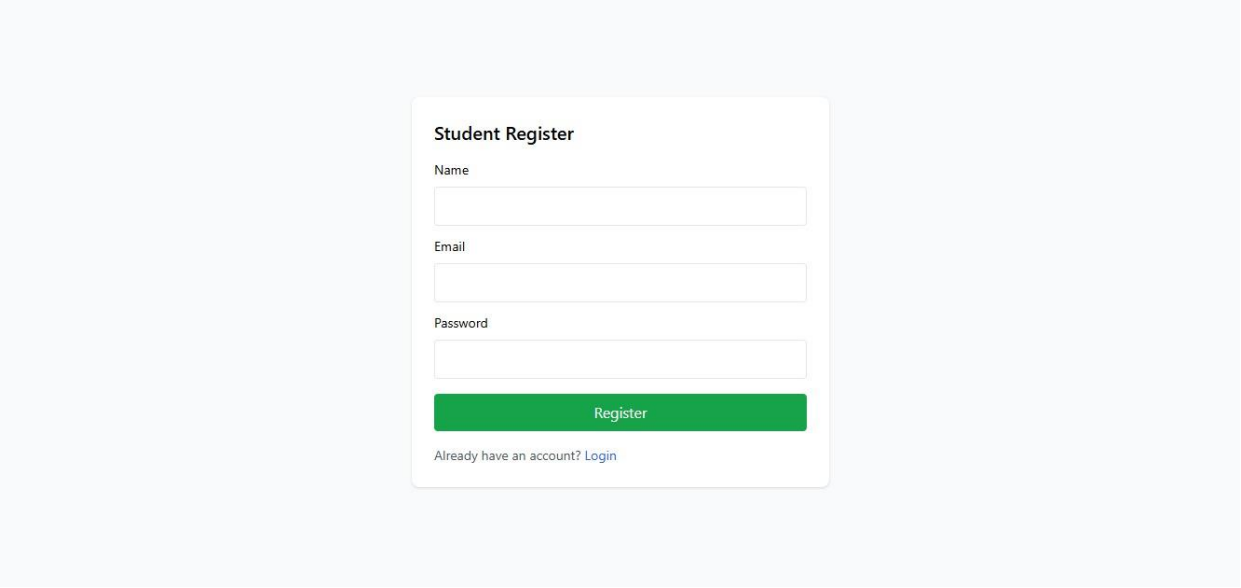

Figure 3. Student Registration Page

4.2 Application Layer — Amazon EC2 (Node.js / Express API)

The backend REST API runs on an Amazon EC2 instance within an Amazon VPC, under PM2, with Nginx acting as a reverse proxy. A dedicated security group attached within the VPC restricts inbound access to only the ports required for SSH management and web traffic.

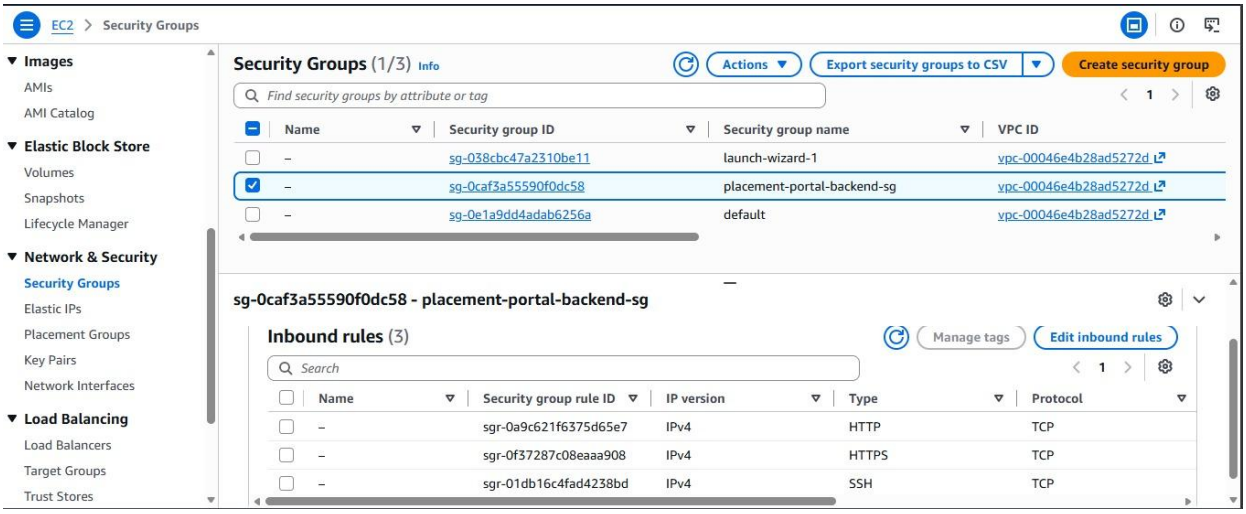

Figure 4. EC2 Backend Security Group — Inbound Rules (SSH, HTTP, HTTPS)

4.3 Data Layer — MongoDB Atlas

Application data is organized into five collections — students, companies, jobs, applications, and admins — each modeled to support fast lookups for dashboards and application-status tracking.

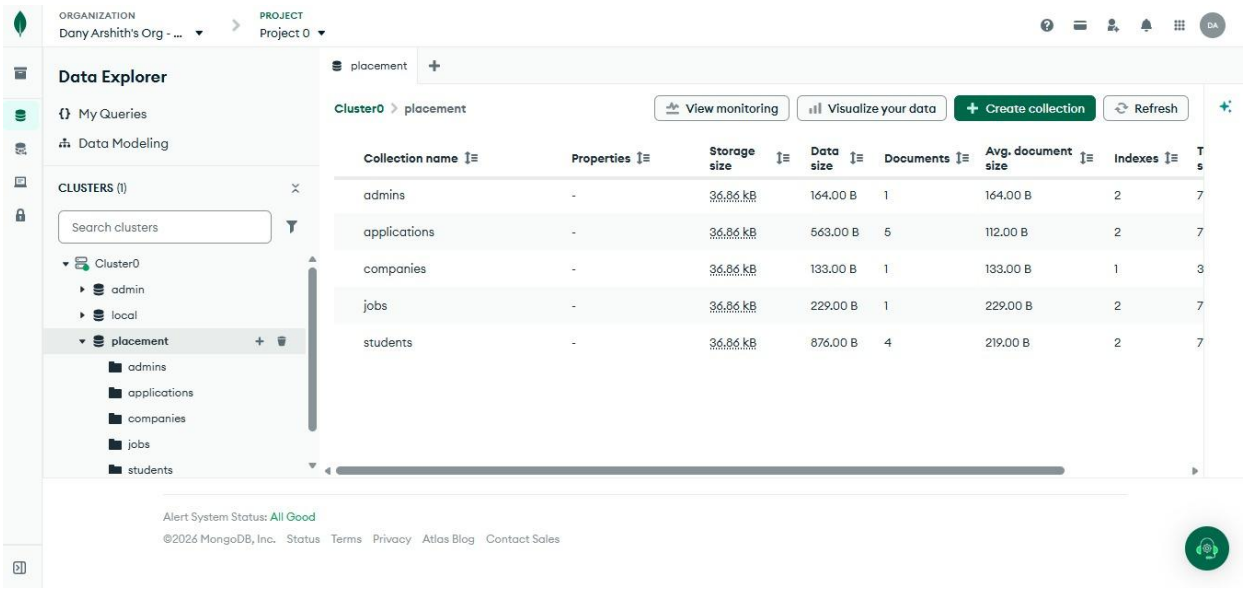

Figure 5. MongoDB Atlas Collections for the Placement Database

Each job posting document stores the company reference, title, description, salary, location, eligibility criteria, and application deadline, allowing the frontend to render rich job listings directly from a single document.

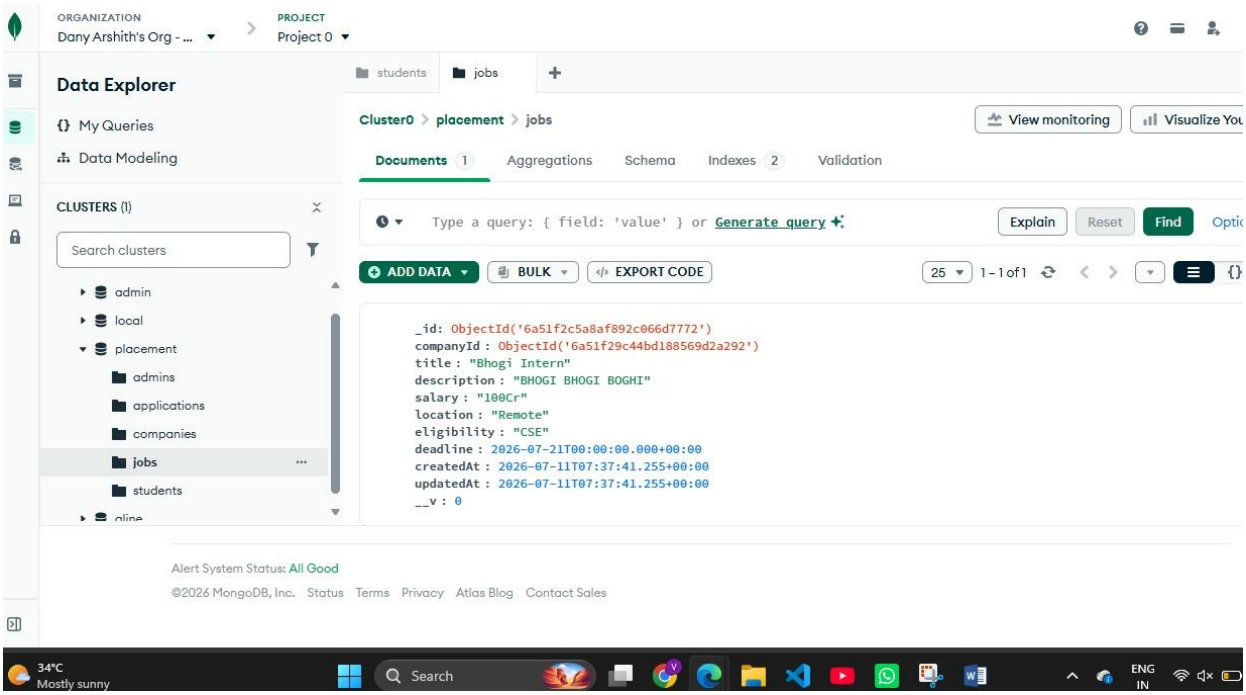

Figure 6. Sample Job Posting Document in the jobs Collection

4.4 Placement Officer / Admin Module

Placement administrators manage companies, job postings, applications, and registered students from a single dashboard, which surfaces summary counts and recent activity at a glance.

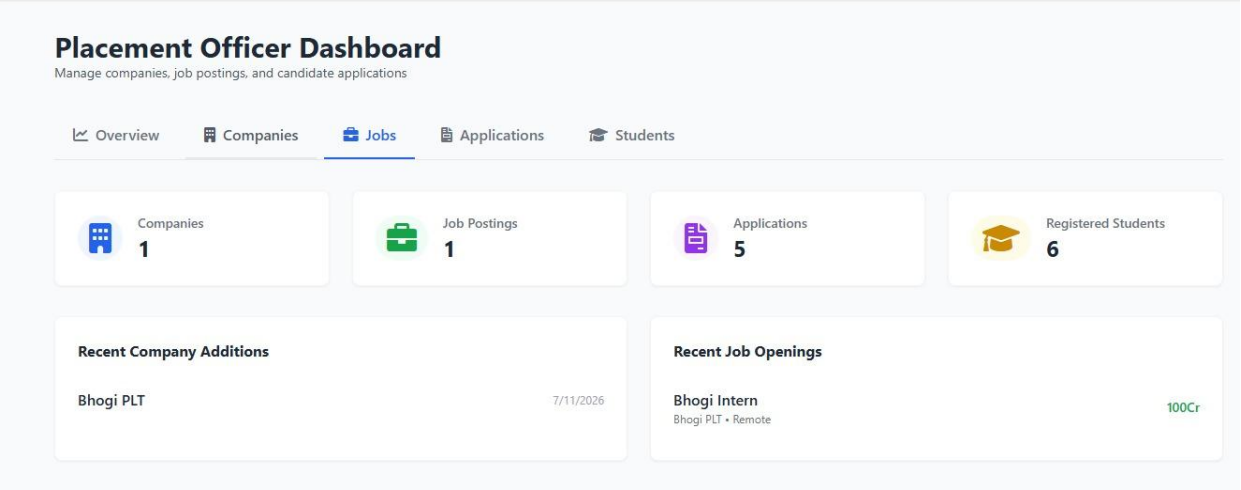

Figure 7. Placement Officer Dashboard — Job Management View

4.5 Student Application Tracking

Students can view their applied jobs and follow a visual progress tracker that moves through Applied, Shortlisted, and Accepted stages as the placement officer updates their status.

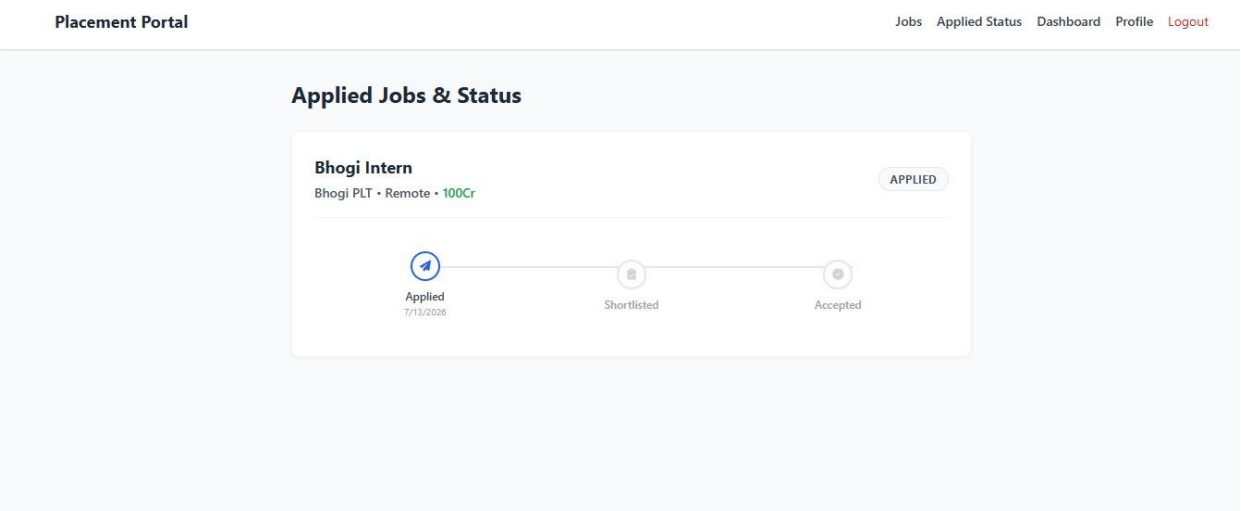

Figure 8. Student View — Applied Jobs and Status Tracker

A personal dashboard summarizes each student's application activity and highlights recently posted jobs relevant to them.

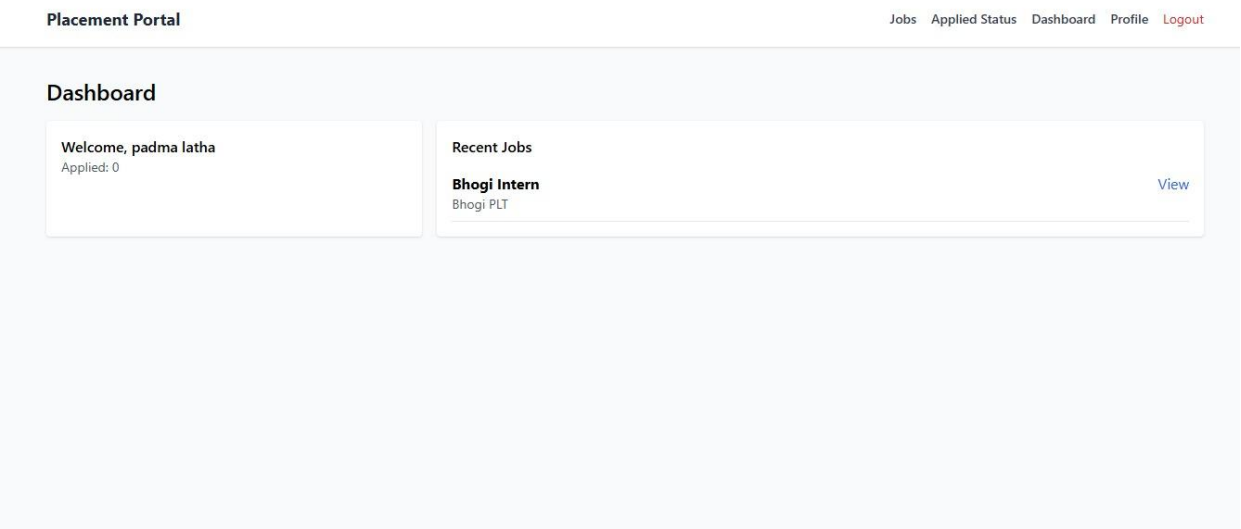

Figure 9. Student Dashboard

4.6 File Storage — Amazon S3

A separate Amazon S3 bucket stores resumes, profile photographs, and company logos uploaded through the portal, keeping large binary files out of the MongoDB database and allowing them to be served or downloaded independently.

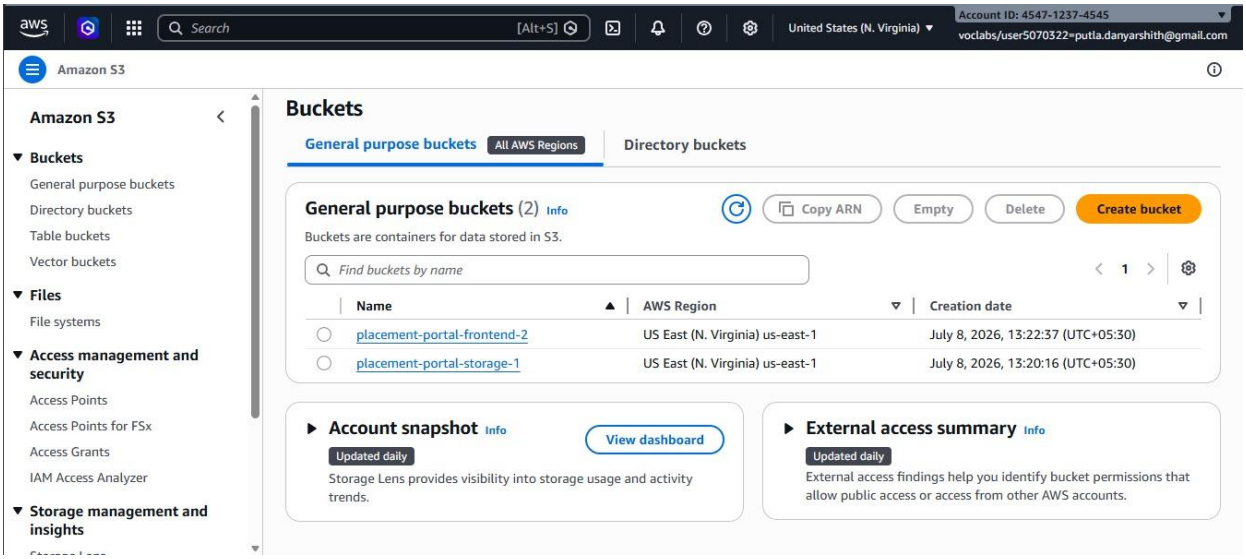

Figure 10. Amazon S3 Buckets — Frontend Hosting and File Storage

4.7 Security — AWS IAM and Amazon VPC

AWS IAM roles attached to the EC2 instance grant the backend only the permissions it needs — reading and writing to the designated S3 bucket and, where applicable, publishing logs — following the principle of least privilege rather than relying on long-lived static credentials embedded in the application. The EC2 instance runs inside an Amazon VPC, which provides an

isolated network environment; a security group attached to the instance controls inbound traffic, allowing only SSH, HTTP, and HTTPS.

5. IMPLEMENTATION

5.1 JWT-Based Session Management

Once authenticated, the backend issues a signed JWT using the HS512 algorithm. Early in development, the signing secret was too short for HS512's minimum key-size requirement, which caused token generation failures; the secret was resized to meet the algorithm's requirements, and JwtUtil and the JwtAuthenticationFilter middleware were updated accordingly to validate tokens on every protected route.

5.2 Resolving Partial Database Commits

A defect in the original AuthService meant that if a JWT key error occurred mid-registration, a partial user record could still be committed to the database, leaving orphaned or incomplete accounts. AuthService was restructured so that user creation and token issuance are treated as a single logical operation — if key validation fails, no database write is committed.

5.3 Endpoint Alignment Between Frontend and Backend

As the student, alumni, and admin dashboards were built out, several frontend components called API paths that did not match the corresponding Express route definitions. Backend route paths for student, alumni, and admin endpoints were audited and corrected to align with the calls made from dashboard and job-listing components, eliminating a class of silent 404 errors.

5.4 File Uploads to Amazon S3

Resume and profile photo uploads are streamed from the Express backend to the dedicated Amazon S3 bucket, with the resulting object URL stored on the corresponding student or company document in MongoDB rather than storing the file itself in the database.

5.5 Deployment Pipeline

The production build of the HTML, CSS, and JavaScript frontend is uploaded to the S3 static website bucket, while the backend is deployed to EC2, kept alive under PM2, and proxied through Nginx. IAM roles scope the EC2 instance's permissions to exactly the AWS resources it needs.

6. RESULTS

6.1 Functional Multi-Role Workflows

The deployed portal supports complete end-to-end workflows for all three roles: students can register, browse jobs, apply with a resume, and track status; placement officers can add companies, publish jobs, and review applications; and alumni can view placement activity through their own dashboard.

6.2 Reliable Authentication

After resizing the JWT signing secret and restructuring AuthService, registration and login succeeded consistently across repeated test runs, with no further partial-commit errors observed.

6.3 Consistent Frontend-Backend Integration

Following the endpoint audit, all dashboard, job-listing, and application-status components successfully called their corresponding backend routes, removing the mismatches that had previously produced broken pages.

6.4 Cloud-Hosted, Scalable Deployment

With the frontend served from Amazon S3 and the backend running on Amazon EC2 behind Nginx, the portal is accessible as a standard web application without any locally hosted infrastructure, and file storage on S3 keeps the MongoDB Atlas database lean and fast.

7. API DOCUMENTATION

This section provides a complete reference of the College Placement Portal's REST API, covering every endpoint, its request parameters, and its response format. The API follows a resource-oriented design with JWT-based authentication, and is organized into six functional groups: general system information, authentication, student management, job operations, company management, and administrative operations.

7.1 General API Information

Base URL

By default, the backend API is served at http://localhost:4000 (or the port configured via the PORT environment variable). All routes except /health are prefixed with /api, giving a base endpoint of http://localhost:4000/api.

Authentication & Authorization

The API uses JSON Web Tokens (JWT) to secure protected endpoints. Clients must include the token in the Authorization header as a Bearer token:

```
Authorization: Bearer <your_jwt_token>
```

Two roles are recognized by the API:

- Student — access to profile management, file uploads, job search, and job applications.
- Admin — full access to create, update, and delete companies, jobs, and student profiles, and to update application statuses.

Rate Limiting

Authentication endpoints under /api/auth/* are protected by a sliding-window rate limiter, allowing a maximum of 15 requests per 15-minute window per IP address. Requests beyond this limit receive:

```
HTTP 429 Too Many Requests
{
  "message": "Too many requests. Please try again later."
}
```

File Upload Constraints

All file uploads use multipart/form-data, mediated through Multer, and are stored in AWS S3 (or a mock fallback store in development). Allowed MIME types are application/pdf, image/jpeg, and image/png, with a maximum file size of 5 MB. Uploads that fail validation return HTTP 500 with a message such as "Invalid file type".

7.2 Global System Endpoints

Get Server Health

Verifies that the backend server is running and reports the current operating mode.

Route: `/health  (not prefixed by /api)` **Method:** GET **Auth:** *Public*

Success Response (200 OK)

```
{
  "status": "ok",
  "phase": 3,
  "mode": "mock-backend"
}
```

7.3 Auth Management — /api/auth

Student Registration

Registers a new student account. On success, returns a JWT and the newly created student object.

Route: `/api/auth/register` **Method:** POST **Auth:** *Public (rate-limited)*

Request Body (JSON)

Field	Type	Required	Description
name	String	Yes	Student's full name.
email	String	Yes	Must be a valid email format. Must be unique.

Field	Type	Required	Description
password	String	Yes	Password, minimum 6 characters.

Example Request Body

```
{
  "name": "John Doe",
  "email": "john@example.com",
  "password": "johnpassword"
}
```

Success Response (201 Created)

```
{
  "token": "eyJhbGciOiJIUzI1NiIsIn...",
  "user": {
    "id": "student_x1y2z3",
    "name": "John Doe",
    "email": "john@example.com",
    "phone": "",
    "branch": "",
    "cgpa": null,
    "skills": [],
    "resumeUrl": "",
    "profileImage": "",
    "createdAt": "2026-07-13T11:22:56.000Z"
  }
}
```

Error Responses

400 Bad Request — validation error:

```
{
  "errors": [
    { "type": "field", "value": "123", "msg": "Invalid value", "path":
"password", "location": "body" }
  ]
}
```

400 Bad Request — email already registered:

```
{
  "message": "Email already registered"
}
```

Student Login

Authenticates a registered student and returns a JWT.

Route: `/api/auth/login` **Method:** POST **Auth:** *Public (rate-limited)*

Request Body (JSON)

Field	Type	Required	Description
email	String	Yes	Must be a valid email format.
password	String	Yes	Student's password.

Example Request Body

```
{
  "email": "alice@example.com",
  "password": "student123"
}
```

Success Response (200 OK)

```
{
  "token": "eyJhbGciOiJIUzI1NiIsIn...",
  "user": {
    "id": "student_1",
    "name": "Alice Student",
    "email": "alice@example.com",
    "phone": "9876543210",
    "branch": "Computer Science",
    "cgpa": 8.7,
    "skills": ["JavaScript", "Node.js", "React"],
    "resumeUrl": "",
    "profileImage": "",
    "createdAt": "2026-07-13T05:54:19.000Z"
  }
}
```

Error Responses

401 Unauthorized — invalid credentials:

```
{
  "message": "Invalid credentials"
}
```

Admin Login

Authenticates an administrative / placement-officer user.

Route: `/api/auth/admin/login` **Method:** POST **Auth:** *Public (rate-limited)*

Request Body (JSON)

Field	Type	Required	Description
username	String	Yes	Admin username.
password	String	Yes	Admin password.

Example Request Body

```
{
  "username": "admin",
  "password": "admin123"
}
```

Success Response (200 OK)

```
{
  "token": "eyJhbGciOiJIUzI1NiIsIn...",
  "user": {
    "id": "admin_1",
    "username": "admin",
    "role": "placement-officer"
  }
}
```

Error Responses

401 Unauthorized:

```
{
  "message": "Invalid credentials"
```

```
}
```

7.4 Student Management — /api/student

Get Current Student Profile

Retrieves details of the authenticated student.

Route: `/api/student/profile` **Method:** GET **Auth:** *Student Bearer Token*

Success Response (200 OK)

```
{
  "id": "student_1",
  "name": "Alice Student",
  "email": "alice@example.com",
  "phone": "9876543210",
  "branch": "Computer Science",
  "cgpa": 8.7,
  "skills": ["JavaScript", "Node.js", "React"],
  "resumeUrl": "",
  "profileImage": "",
  "createdAt": "2026-07-13T05:54:19.000Z"
}
```

Update Student Profile

Updates partial fields of the authenticated student's profile.

Route: `/api/student/profile` **Method:** PUT **Auth:** *Student Bearer Token*

Request Body (JSON) — optional keys

Field	Type	Required	Description
phone	String	No	Contact number.
branch	String	No	Field of study / branch.
cgpa	Number	No	Grade point average.
skills	Array<String>	No	Technical / soft skills list.

Example Request Body

```
{
  "phone": "9999988888",
```

```
  "branch": "Information Technology",
  "cgpa": 9.1,
  "skills": ["Node.js", "Docker", "AWS"]
}
```

Success Response (200 OK)

```
{
  "id": "student_1",
  "name": "Alice Student",
  "email": "alice@example.com",
  "phone": "9999988888",
  "branch": "Information Technology",
  "cgpa": 9.1,
  "skills": ["Node.js", "Docker", "AWS"],
  "resumeUrl": "",
  "profileImage": "",
  "createdAt": "2026-07-13T05:54:19.000Z"
}
```

Upload Resume

Uploads a resume PDF to S3. The file is scanned during processing and the resulting object key is saved to the student's profile.

Route: `/api/student/upload-resume` **Method:** POST **Auth:** *Student Bearer Token*

Request Body — multipart/form-data

Form field name: resume (binary PDF file).

Success Response (200 OK)

```
{
  "url":                    "https://s3.us-east-1.amazonaws.com/mock-
bucket/resumes/1715560000_resume.pdf"
}
```

Upload Profile Photo

Uploads a student portrait image to S3.

Route: `/api/student/upload-photo` **Method:** POST **Auth:** *Student Bearer Token*

Request Body — multipart/form-data

Form field name: photo (binary JPEG/PNG file).

Success Response (200 OK)


```
{
  "url":                    "https://s3.us-east-1.amazonaws.com/mock-
bucket/photos/1715560000_photo.png"
}
```


7.5 Job Operations — /api/jobs

Get All Jobs (Search / Pagination)

Returns a list of jobs matching the given filters. Each job includes its nested Company details.

Route: `/api/jobs` **Method:** GET **Auth:** *Public*

Query Parameters

Field	Type	Required	Description
search	String	No	Filter by job title, location, or description.
page	Integer	No	Page index. Default 1.
limit	Integer	No	Jobs per page. Default 10.

Success Response (200 OK)


```
{
  "data": [
    {
      "id": "job_1",
      "companyId": "company_1",
      "title": "Junior Backend Engineer",
      "description": "Build and maintain REST APIs.",
      "salary": "4 LPA",
      "location": "Remote",
      "eligibility": "CS, IT",
      "deadline": "2026-07-31T00:00:00.000Z",
      "createdAt": "2026-07-13T05:54:19.000Z",
      "company": {
        "id": "company_1",
        "companyName": "Acme Corp",
        "description": "Engineering and manufacturing",
        "website": "https://acme.example.com",
        "logo": "",
        "createdAt": "2026-07-13T05:54:19.000Z"
      }
```

```
    }
  ],
  "meta": { "total": 1, "page": 1, "limit": 5 }
}
```

Get Job By ID

Returns full details of a specific job.

Route: `/api/jobs/:id` **Method:** GET **Auth:** *Public*

Success Response (200 OK)

```
{
  "id": "job_1",
  "companyId": "company_1",
  "title": "Junior Backend Engineer",
  "description": "Build and maintain REST APIs.",
  "salary": "4 LPA",
  "location": "Remote",
  "eligibility": "CS, IT",
  "deadline": "2026-07-31T00:00:00.000Z",
  "createdAt": "2026-07-13T05:54:19.000Z",
  "company": { "id": "company_1", "companyName": "Acme Corp" }
}
```

Error Responses

404 Not Found:

```
{
  "message": "Not found"
}
```

Apply for Job

Submits a student's application for a job opening.

Route: `/api/jobs/apply` **Method:** POST **Auth:** *Student Bearer Token*

Request Body (JSON)

Field	Type	Required	Description
jobId	String	Yes	ID of the job to apply to.

Example Request Body

```
{
  "jobId": "job_2"
}
```

Success Response (200 OK)

```
{
  "id": "app_2",
  "studentId": "student_1",
  "jobId": "job_2",
  "status": "applied",
  "notes": "",
  "appliedAt": "2026-07-13T11:22:56.000Z"
}
```

Error Responses

400 Bad Request — already applied:

```
{
  "message": "Already applied"
}
```

404 Not Found — job not found:

```
{
  "message": "Job not found"
}
```

Get Applied Jobs

Lists all job applications submitted by the authenticated student, including nested job and company details.

Route: `/api/jobs/applied` **Method:** **GET** **Auth:** *Student Bearer Token*

Success Response (200 OK)

```
{
  "id": "app_1",
  "studentId": "student_1",
  "jobId": "job_1",
```

```
  "status": "applied",
  "notes": "",
  "appliedAt": "2026-07-13T05:54:19.000Z",
  "job": { "id": "job_1", "title": "Junior Backend Engineer" },
  "company": { "id": "company_1", "companyName": "Acme Corp" }
}
```

7.6 Company Management — /api/company

Create Company

Registers a new recruiting company.

Route: `/api/company` **Method:** POST **Auth:** *Admin Bearer Token*

Request Body — multipart/form-data

Field	Type	Required	Description
companyName	String	Yes	Name of the organization.
description	String	No	Corporate description.
website	String	No	URL of corporate website.
logo	File	No	Binary logo image file.

Success Response (201 Created)

```
{
  "id": "company_3",
  "companyName": "Global Tech Services",
  "description": "A global software solutions provider.",
  "website": "https://globaltech.example.com",
  "logo":                  "https://s3.us-east-1.amazonaws.com/mock-
bucket/logos/1715560000_logo.png",
  "createdAt": "2026-07-13T11:22:56.000Z"
}
```

Get All Companies

Lists all registered companies.

Route: `/api/company` **Method:** GET **Auth:** *Admin Bearer Token*

Success Response (200 OK)

```
[
  {
    "id": "company_1",
    "companyName": "Acme Corp",
    "description": "Engineering and manufacturing",
    "website": "https://acme.example.com",
    "logo": "",
    "createdAt": "2026-07-13T05:54:19.000Z"
  }
]
```


Update Company

Updates the details of a company. Partial updates are accepted.

Route: `/api/company/:id` **Method:** PUT **Auth:** *Admin Bearer Token*

Example Request Body


```
{
  "description": "An updated description for Acme Corp."
}
```


Success Response (200 OK)


```
{
  "id": "company_1",
  "companyName": "Acme Corp",
  "description": "An updated description for Acme Corp.",
  "website": "https://acme.example.com",
  "logo": "",
  "createdAt": "2026-07-13T05:54:19.000Z"
}
```


Delete Company

Removes a company from the portal.

Route: `/api/company/:id` **Method:** DELETE **Auth:** *Admin Bearer Token*

Success Response (200 OK)


```
{
  "message": "Deleted"
}
```

7.7 Admin Management — /api/admin

Create Placement Job

Posts a new job opening for students to apply to.

Route: `/api/admin/job` **Method:** POST **Auth:** *Admin Bearer Token*

Request Body (JSON)

Field	Type	Required	Description
companyId	String	Yes	ID of the registered company hosting the job.
title	String	Yes	Job designation.
description	String	No	Detailed roles and responsibilities.
salary	String	No	e.g. "8 LPA" or "$80,000".
location	String	No	e.g. "Hybrid", "Remote", "Onsite".
eligibility	String	No	Qualification requirements, e.g. "CS, IT, ECE".
deadline	Date	No	Application deadline timestamp.

Example Request Body

```
{
  "companyId": "company_1",
  "title": "Senior QA Automation Engineer",
  "description": "Create test automation framework for backend API.",
  "salary": "8 LPA",
  "location": "Hybrid",
  "eligibility": "CS, IT, ECE",
  "deadline": "2026-09-30T00:00:00.000Z"
}
```

Success Response (201 Created)

```
{
  "id": "job_3",
  "companyId": "company_1",
  "title": "Senior QA Automation Engineer",
  "description": "Create test automation framework for backend API.",
  "salary": "8 LPA",
```

```
  "location": "Hybrid",
  "eligibility": "CS, IT, ECE",
  "deadline": "2026-09-30T00:00:00.000Z",
  "createdAt": "2026-07-13T11:22:56.000Z"
}
```

Error Responses

400 Bad Request — invalid companyId:

```
{
  "message": "Invalid companyId"
}
```

Update Placement Job

Edits the details of a posted job. Partial updates are accepted.

Route: `/api/admin/job/:id` **Method:** **PUT** **Auth:** *Admin Bearer Token*

Example Request Body

```
{
  "salary": "6 LPA"
}
```

Success Response (200 OK)

```
{
  "id": "job_1",
  "companyId": "company_1",
  "title": "Junior Backend Engineer",
  "salary": "6 LPA",
  "location": "Remote",
  "eligibility": "CS, IT",
  "deadline": "2026-07-31T00:00:00.000Z",
  "createdAt": "2026-07-13T05:54:19.000Z"
}
```

Delete Placement Job

Removes a posted job.

Route: `/api/admin/job/:id` **Method:** **DELETE** **Auth:** *Admin Bearer Token*

Success Response (200 OK)

```
{
  "message": "Deleted"
}
```

List All Students

Returns details of all registered students.

Route: `/api/admin/students` **Method:** `GET` **Auth:** *Admin Bearer Token*

Success Response (200 OK)

```
[
  {
    "id": "student_1",
    "name": "Alice Student",
    "email": "alice@example.com",
    "phone": "9876543210",
    "branch": "Computer Science",
    "cgpa": 8.7,
    "skills": ["JavaScript", "Node.js", "React"]
  }
]
```

Update Student Profile (Administrative Override)

Allows an administrator to modify a student's information — for example, verifying CGPA or correcting a profile.

Route: `/api/admin/student/:id` **Method:** `PUT` **Auth:** *Admin Bearer Token*

Accepts the same fields as PUT /api/student/profile.

Success Response (200 OK)

```
{
  "id": "student_1",
  "name": "Alice Student Override",
  "email": "alice@example.com",
  "cgpa": 9.0,
  "skills": ["JavaScript", "Node.js", "React"]
}
```

Delete Student

Removes a student's record and account from the system.

Route: `/api/admin/student/:id` **Method:** **DELETE** **Auth:** *Admin Bearer Token*

Success Response (200 OK)

```
{
  "message": "Deleted"
}
```

List All Applications

Lists every job application in the portal, populated with the related student, job, and company documents.

Route: `/api/admin/applications` **Method:** **GET** **Auth:** *Admin Bearer Token*

Success Response (200 OK)

```
[
  {
    "id": "app_1",
    "studentId": "student_1",
    "jobId": "job_1",
    "status": "applied",
    "appliedAt": "2026-07-13T05:54:19.000Z",
    "student": { "id": "student_1", "name": "Alice Student" },
    "job": { "id": "job_1", "title": "Junior Backend Engineer" },
    "company": { "id": "company_1", "companyName": "Acme Corp" }
  }
]
```

Update Application Status

Moves an application through the pipeline — accept, reject, or shortlist a candidate.

Route: `/api/admin/application/:id/status` **Method:** **PUT** **Auth:** *Admin Bearer Token*

Request Body (JSON)

Field	Type	Required	Description
status	String	Yes	Must be one of: applied, shortlisted, rejected, accepted.

Example Request Body

```
{
  "status": "shortlisted"
}
```

Success Response (200 OK)

```
{
  "id": "app_1",
  "studentId": "student_1",
  "jobId": "job_1",
  "status": "shortlisted",
  "notes": "",
  "appliedAt": "2026-07-13T05:54:19.000Z"
}
```

Error Responses

400 Bad Request — invalid status value:

```
{
  "message": "Invalid status"
}
```

404 Not Found:

```
{
  "message": "Application not found"
}
```

7.8 Summary of Endpoints

The table below summarizes every endpoint exposed by the College Placement Portal API.

Route	Method	Description
/health	GET	Server health / mode check
/api/auth/register	POST	Register a new student
/api/auth/login	POST	Student login
/api/auth/admin/login	POST	Admin login

Route	Method	Description
/api/student/profile	GET	Get current student profile
/api/student/profile	PUT	Update student profile
/api/student/upload-resume	POST	Upload resume to S3
/api/student/upload-photo	POST	Upload profile photo to S3
/api/jobs	GET	List / search jobs (paginated)
/api/jobs/:id	GET	Get a single job by ID
/api/jobs/apply	POST	Apply for a job
/api/jobs/applied	GET	List the student's applied jobs
/api/company	POST	Create a company
/api/company	GET	List all companies
/api/company/:id	PUT	Update a company
/api/company/:id	DELETE	Delete a company
/api/admin/job	POST	Create a job posting
/api/admin/job/:id	PUT	Update a job posting
/api/admin/job/:id	DELETE	Delete a job posting
/api/admin/students	GET	List all students
/api/admin/student/:id	PUT	Admin override of a student profile
/api/admin/student/:id	DELETE	Delete a student
/api/admin/applications	GET	List all applications
/api/admin/application/:id/status	PUT	Update application status

8. CONCLUSION

The College Placement Portal demonstrates a complete, cloud-deployed, multi-role recruitment management system built on HTML, CSS, and JavaScript, Node.js/Express, and MongoDB Atlas, hosted on AWS using Amazon S3, Amazon EC2, IAM, and Amazon VPC. The project moved from initial requirements through iterative implementation and debugging — resolving constructor conflicts, endpoint mismatches, JWT key-sizing issues, and partial-commit bugs — to a stable system supporting user registration, resume-based job applications, real-time status tracking, and administrative company and job management.

Beyond its current functionality, the architecture leaves room for future enhancements such as automated interview scheduling, analytics dashboards for placement trends, integration with Amazon SES for higher-volume email delivery, and Amazon CloudFront for faster global content delivery of the frontend. Overall, the project illustrates how a conventional full-stack web application can be adapted to a scalable, secure AWS deployment suited to a real institutional use case.